
CSE D 501

Neural Networks

Taylor Kessler Faulkner

University of Washington

November 12, 2025



Admin

- Reminder: registration for winter course is open!
 - You can enroll in CSE D 502 (CV and Deep Learning) using SLN code 13149
 - More information in message from Kate on Modern AI Methods Students Ed board

Quick reminder on weights

- When can we use weights to assign relative importance to features?
 - When the inputs are standardized
AND
 - When we have controlled for correlated features
 - Remember, highly correlated features (like #bathrooms and #toilets) can share weight

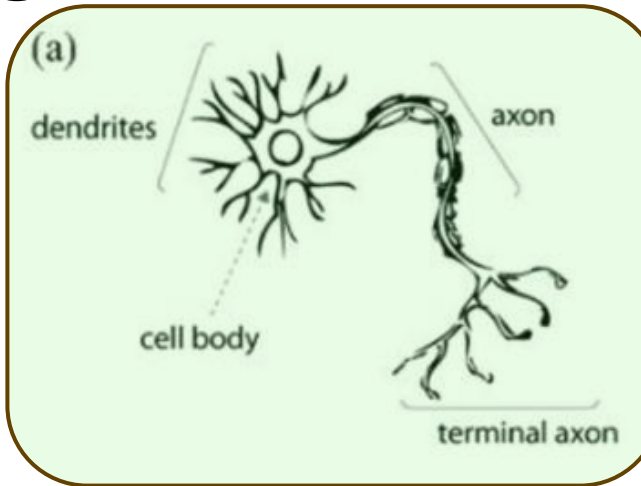
Today's Class

- Neural Networks
- Break
- Neural Networks Cont'd
- Break
- Demo, Discussion

Neural Networks

- What is a neural network?
- Big idea: what if we tried to model a machine learning method after the human brain?
- Model a **neuron** to make a single decision
- Model connections between neurons to mimic synapses for complex decisions

Brain neuron



Artificial neuron

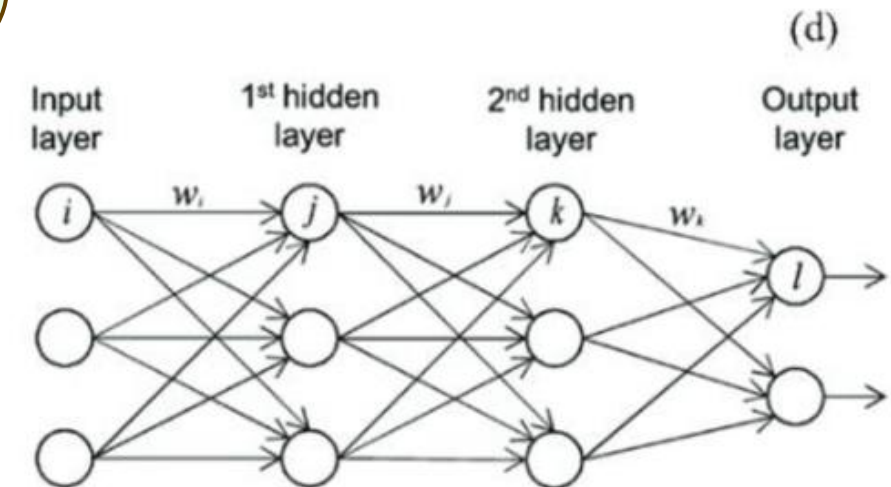
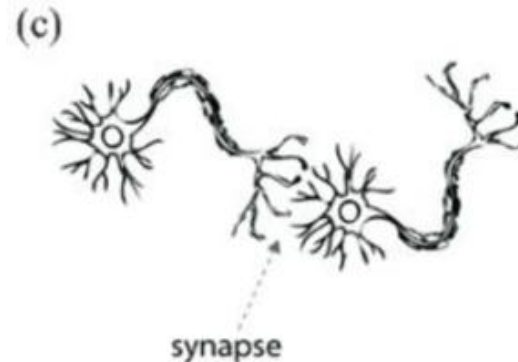
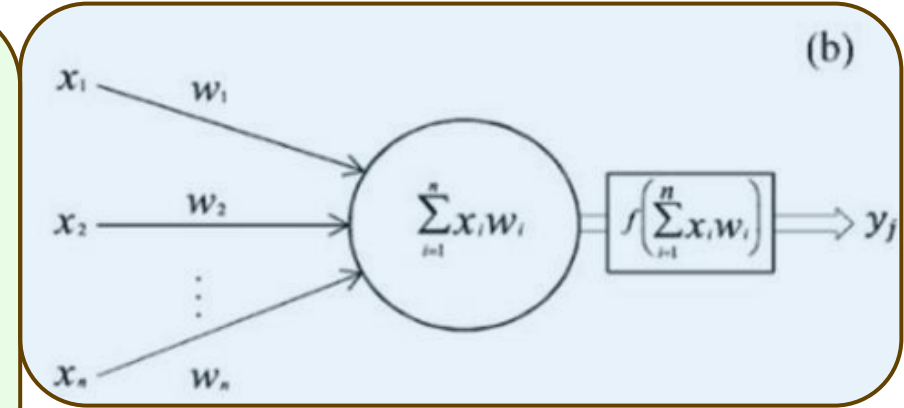


Image from Putra, Sumari & Jamal, Syed Saqib & Abualigah, Laith. (2021). A Novel Deep Learning Pipeline Architecture based on CNN to Detect Covid-19 in Chest X-ray Images. Turkish Journal of Computer and Mathematics Education (TURCOMAT). Vol. 12. 2001-2011.

Neural Networks

- First question: what can we use to mimic brain neurons?
- We discussed **perceptrons** briefly during classification
- By themselves, perceptrons are limited: only linear data, and only binary outputs
- What if we connect many perceptrons together?

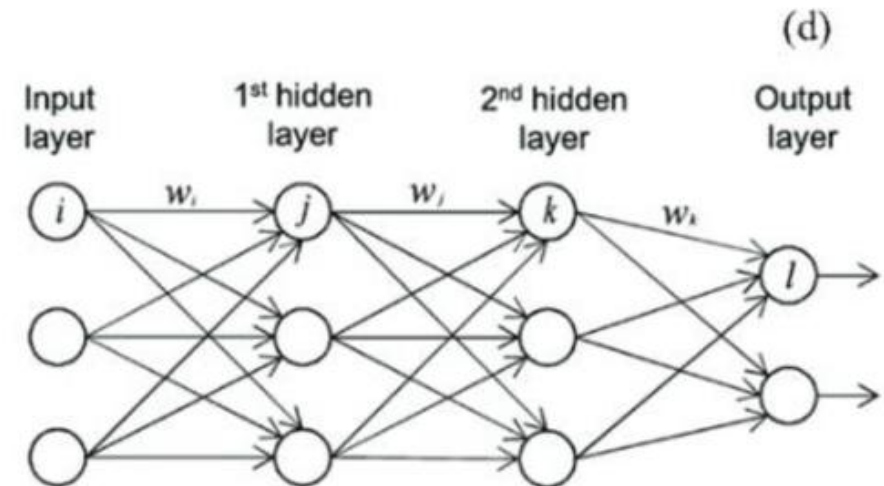
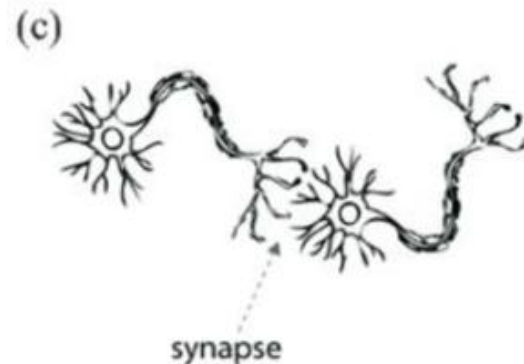
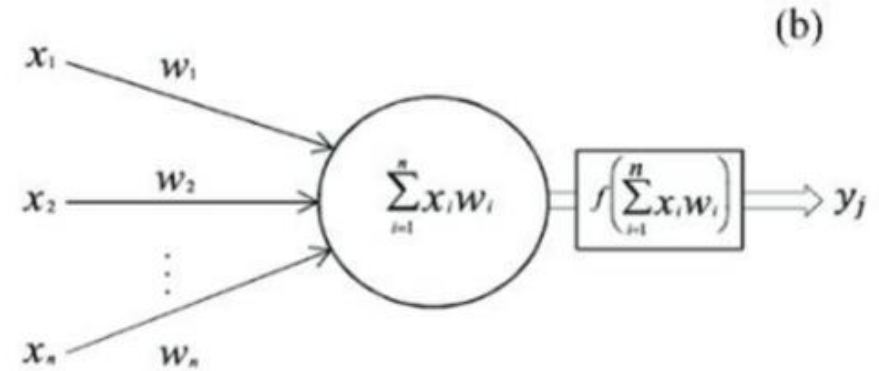
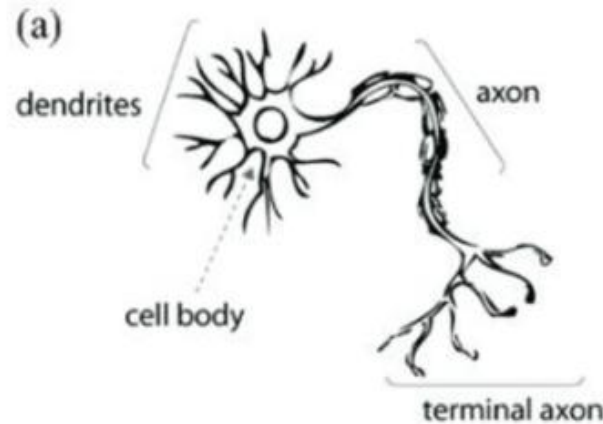


Image from Putra, Sumari & Jamal, Syed Saqib & Abualigah, Laith. (2021). A Novel Deep Learning Pipeline Architecture based on CNN to Detect Covid-19 in Chest X-ray Images. Turkish Journal of Computer and Mathematics Education (TURCOMAT). Vol. 12. 2001-2011.

Reminder: Perceptron Linear Classifier

- The **perceptron** algorithm learns weights by the following pseudocode:

1. Initialize all weights/parameters w to 0 (or small random values)

2. For each $x_i \in \mathcal{X}$

1. Classify x_i : $\hat{y}_i = \begin{cases} +1 & \text{if } \sum_{j=1}^D \hat{w}_j h_j(x_i) \geq 0 \\ -1 & \text{if } \sum_{j=1}^D \hat{w}_j h_j(x_i) < 0 \end{cases}$

2. If $\hat{y}_i = y_i$:

1. Do nothing!

3. If $\hat{y}_i \neq y_i$:

1. Modify the weights using an **update rule**

3. Repeat until convergence (all inputs are classified correctly)

Perceptron Linear Classifier Update Rule

- The **perceptron** update rule is

$$w_j^{(t+1)} = w_j^{(t)} + \eta(y_i - \hat{y}_i)x_i[j]$$

- w_j : the weight of feature j
- $x_i[j]$: the value of feature j for x_i
- η : the step size

If $y_i = \hat{y}_i$: 0

If $y_i \neq \hat{y}_i$: -2 or +2, depending on $y_i = -1$ or 1

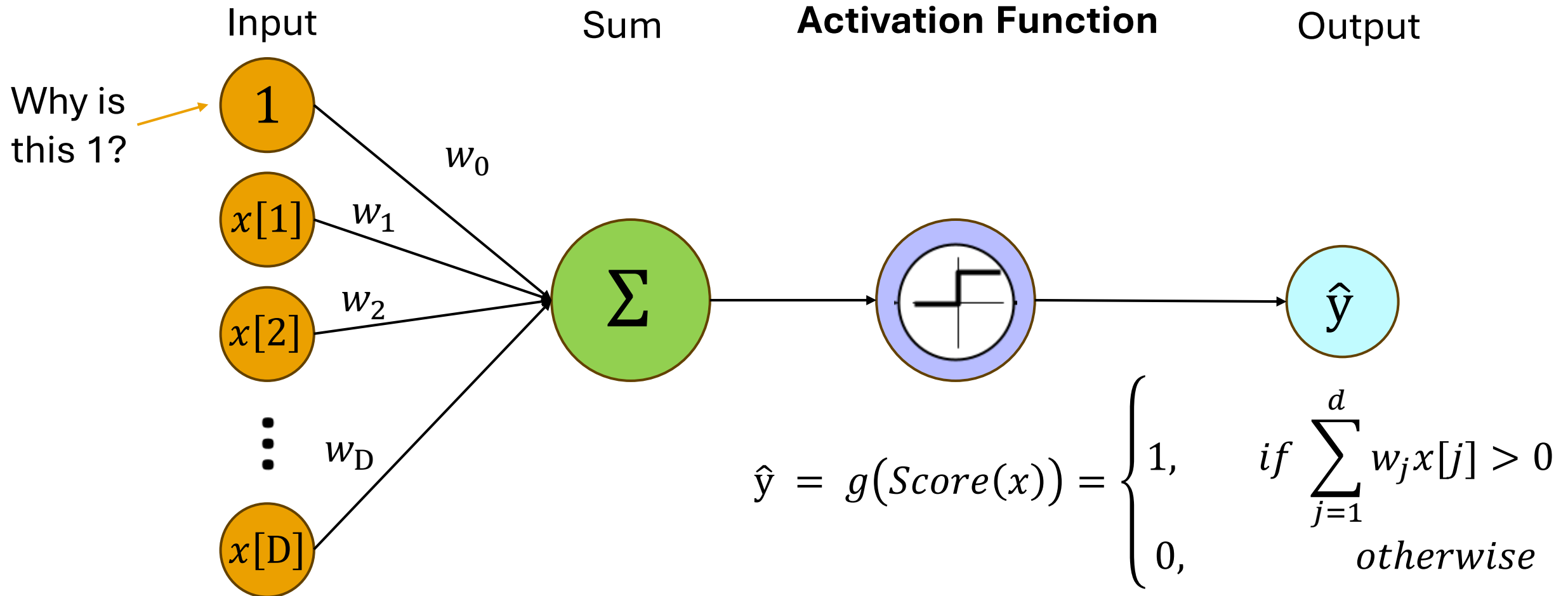
- This can also be written in matrix form

$$w^{(t+1)} = w^{(t)} + \eta(y - \hat{y})x$$

- If η is too small, the algorithm will be very slow, and if η is too large, it may overshoot

Visual representation of Perceptron

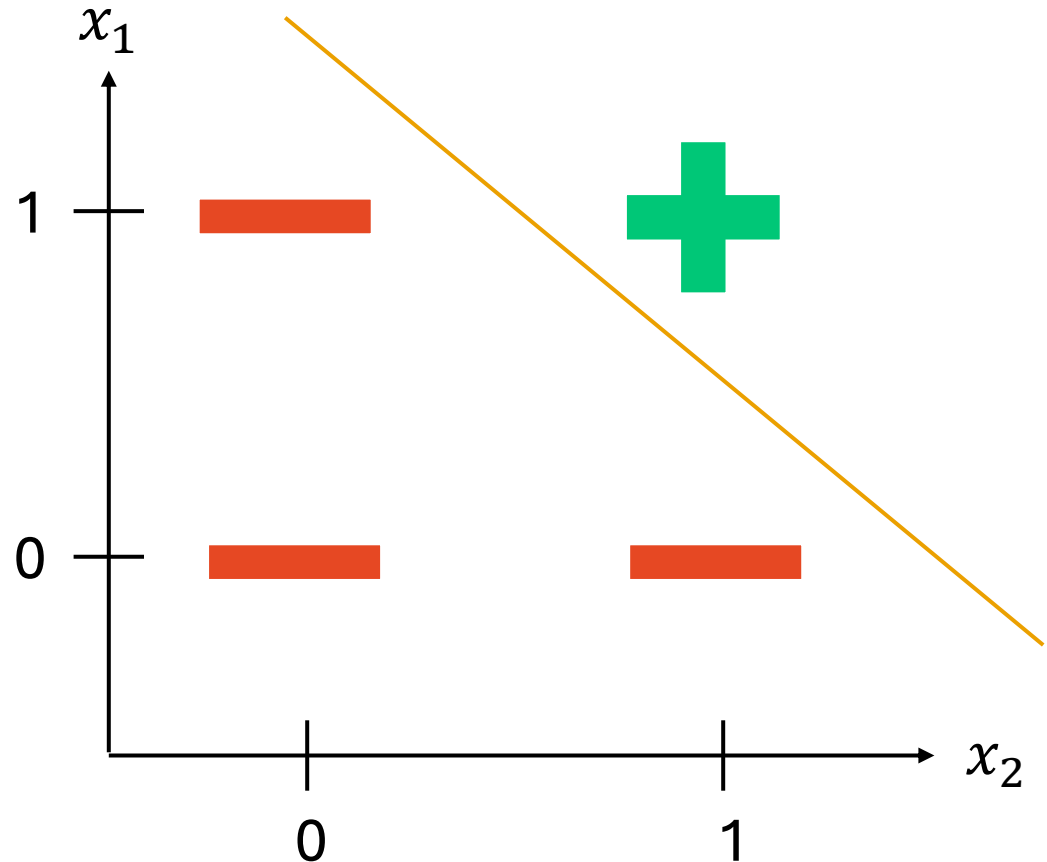
- To more easily talk about perceptrons and neural networks, we will use a **graphical representation**



Non-linear Classification

- Perceptrons can learn many Boolean functions, like AND
- x_1 and x_2 in $[0,1]$
- $\text{AND}(x_1, x_2)$: **1** (true) if and only if x_1 and x_2 are both 1, **-1** (false) otherwise

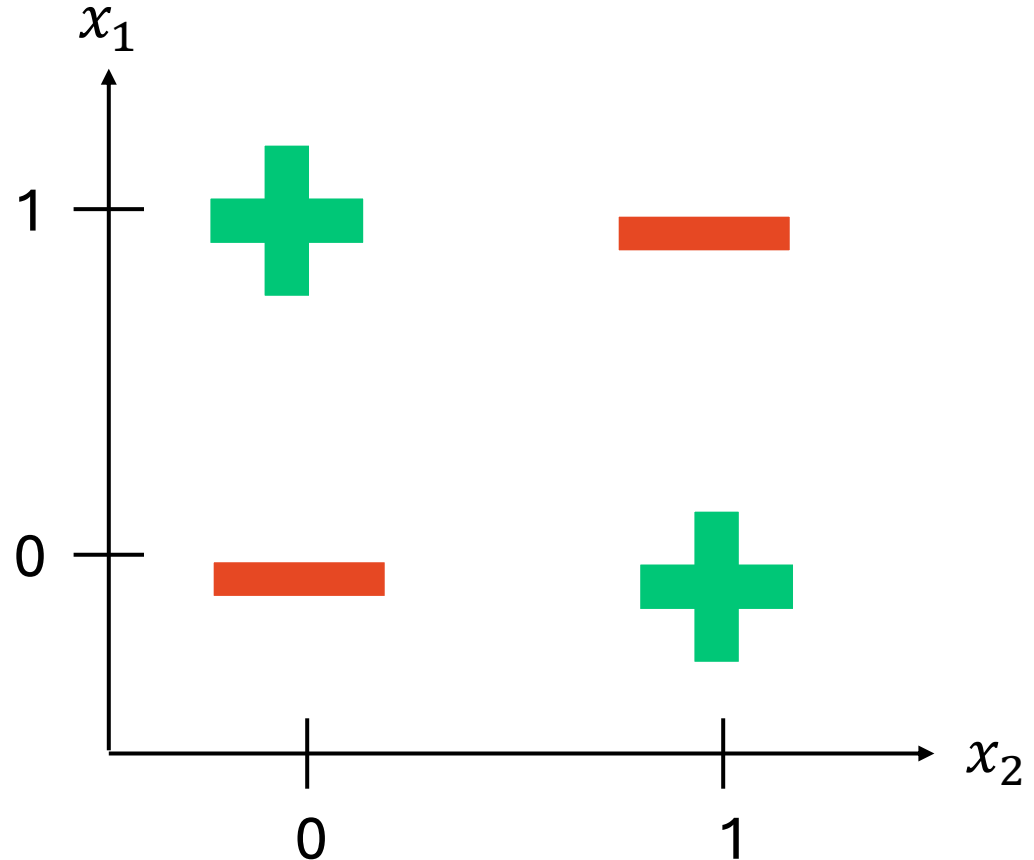
x_1	x_2	y
0	0	-1
0	1	-1
1	0	-1
1	1	+1



Non-linear Classification

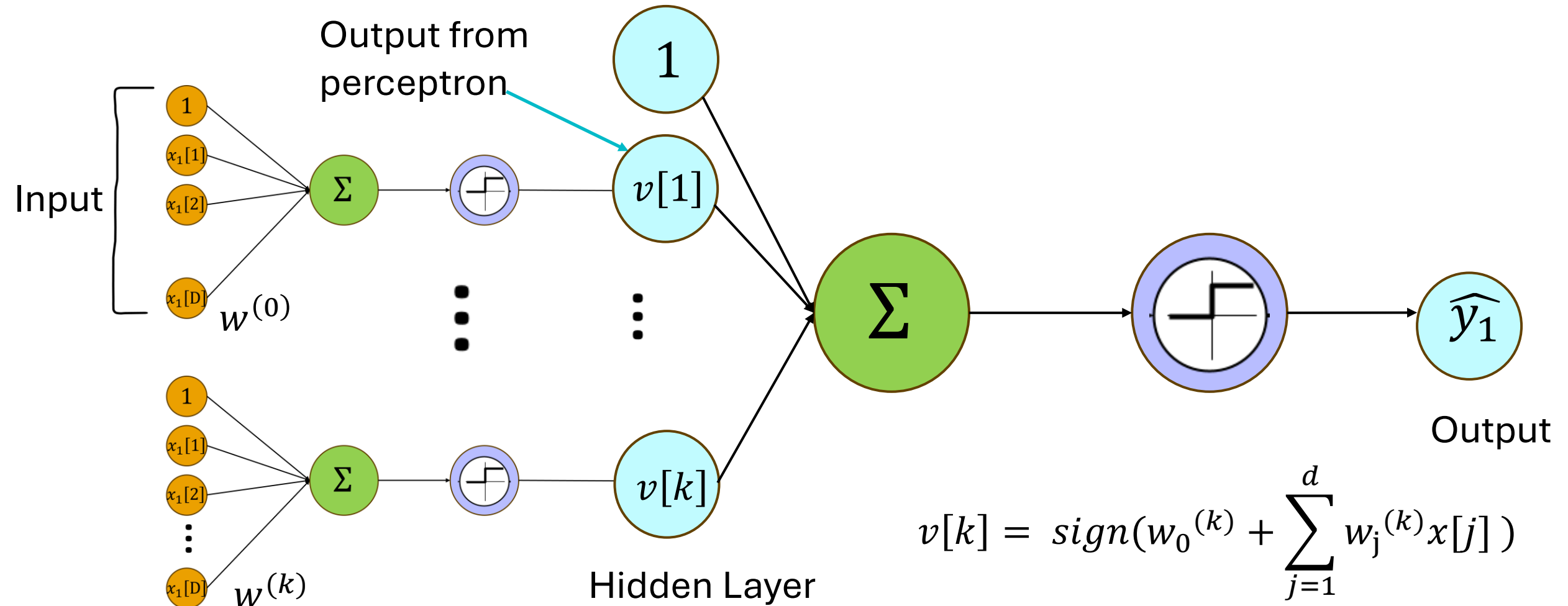
- Perceptrons cannot learn the XOR Boolean function (exclusive or)
- x_1 and x_2 in $[0,1]$
- $\text{XOR}(x_1, x_2)$: **1** (true) if and only if x_1 and x_2 are different, **-1** (false) otherwise

x_1	x_2	y
0	0	-1
0	1	+1
1	0	-1
1	1	+1



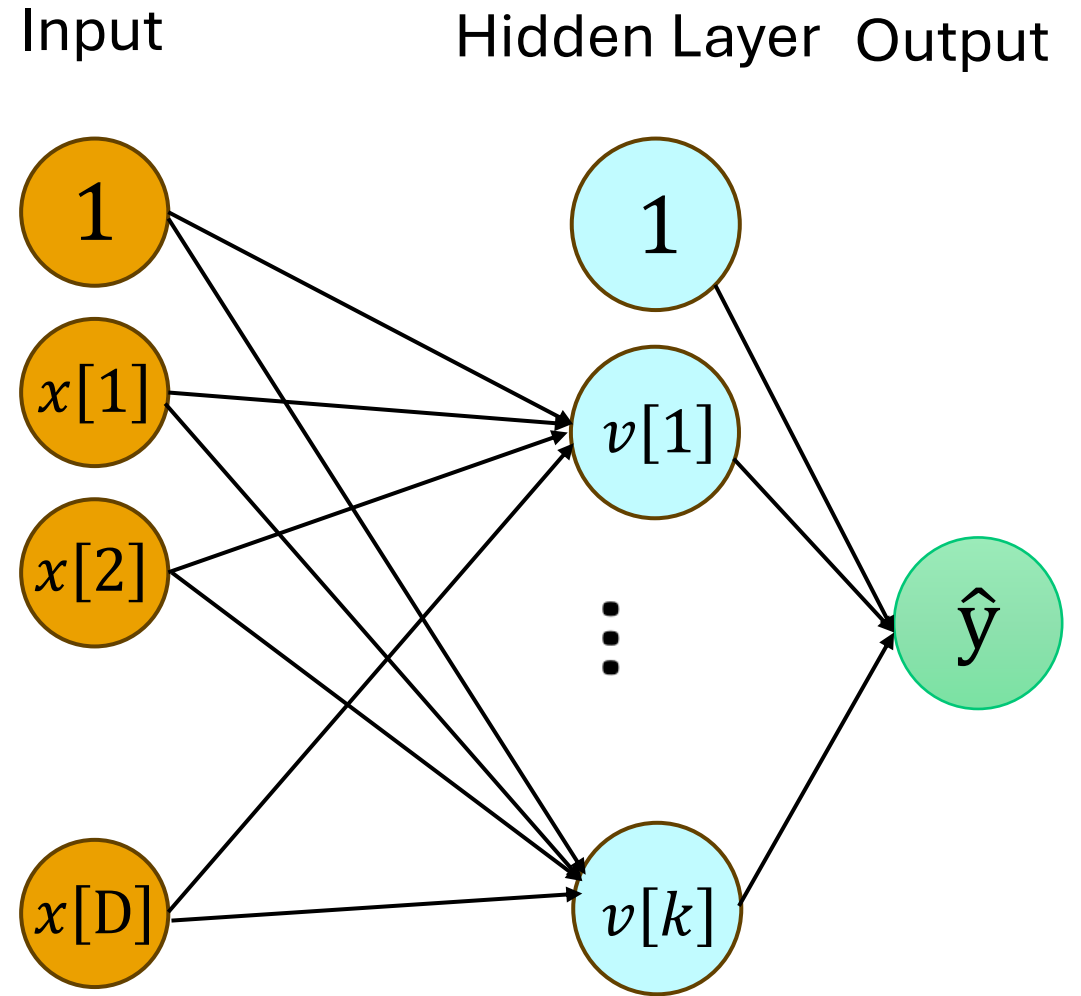
Non-linear Classification

- To classify non-linear data, we have to combine perceptrons



Condensed Graph for Neural Networks

- Because the inputs are the same for every perceptron in the initial layer, we can combine them in the diagram
- The sum and activation function are implicit in each node
- There are weights on **every** arrow



Back to XOR: Exercise

- We can represent

$$x[1] \text{ XOR } x[2] = (x[1] \text{ AND } \neg x[2]) \text{ OR } (\neg x[1] \text{ AND } x[2])$$

- Exercise: for 3 single perceptrons, find the weights needed to compute
 - $(x[1] \text{ AND } \neg x[2])$
 - $(\neg x[1] \text{ AND } x[2])$
 - $(x[1] \text{ OR } x[2])$
- Hints:
 - For the first equation, you're trying to find a weight vector $\mathbf{w}$ such that
$$w[0] + w[1]x[1] + w[2]x[2] \geq 0 \text{ only when } x[1] = 1, x[2] = 0$$
 - Second
$$w[0] + w[1]x[1] + w[2]x[2] \geq 0 \text{ only when } x[1] = 0, x[2] = 1$$
 - Third
$$w[0] + w[1]x[1] + w[2]x[2] \geq 0 \text{ only when } x[1] = 1 \text{ or } x[2] = 1$$
 - Weights will either be ± 0.5 or ± 1

Back to XOR: Exercise

- We can represent

$$x[1] \text{ XOR } x[2] = (x[1] \text{ AND } !x[2]) \text{ OR } (!x[1] \text{ AND } x[2])$$

- Exercise: for 3 single perceptrons, find the weights needed to compute
 - $(x[1] \text{ AND } !x[2])$
 - $(!x[1] \text{ AND } x[2])$
 - $(x[1] \text{ OR } x[2])$
- Weights:
 - $(x[1] \text{ AND } !x[2])$
 - $(!x[1] \text{ AND } x[2])$
 - $(x[1] \text{ OR } x[2])$

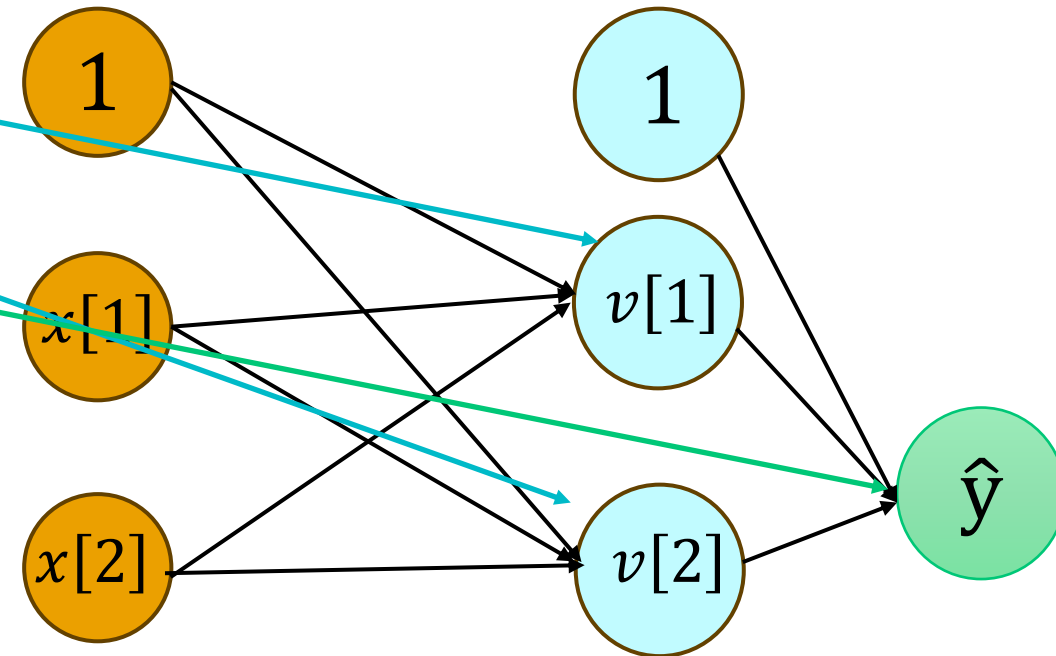
Back to XOR

- With the concept of **hidden layers**, we can now use a neural network to compute the XOR function!

$$x[1] \text{ XOR } x[2] = (x[1] \text{ AND } \neg x[2]) \text{ OR } (\neg x[1] \text{ AND } x[2])$$

- We'll make each node compute one function

- $v[1]: (x[1] \text{ AND } \neg x[2])$
- $v[2]: (\neg x[1] \text{ AND } x[2])$
- $y: (v[1] \text{ OR } v[2])$



Back to XOR

- With the concept of **hidden layers**, we can now use a neural network to compute the XOR function!

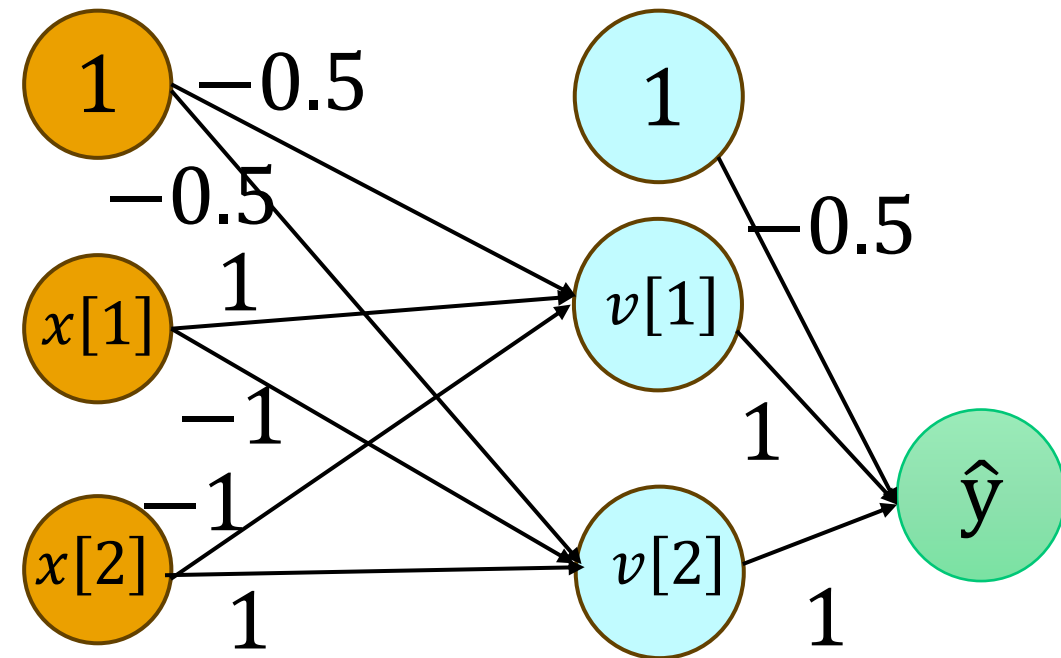
$$x[1] \text{ XOR } x[2] = (x[1] \text{ AND } !x[2]) \text{ OR } (!x[1] \text{ AND } x[2])$$

- We'll make each node compute one function

- $v[1]$: $(x[1] \text{ AND } !x[2])$
- $v[2]$: $(!x[1] \text{ AND } x[2])$
- y : $(v[1] \text{ OR } v[2])$

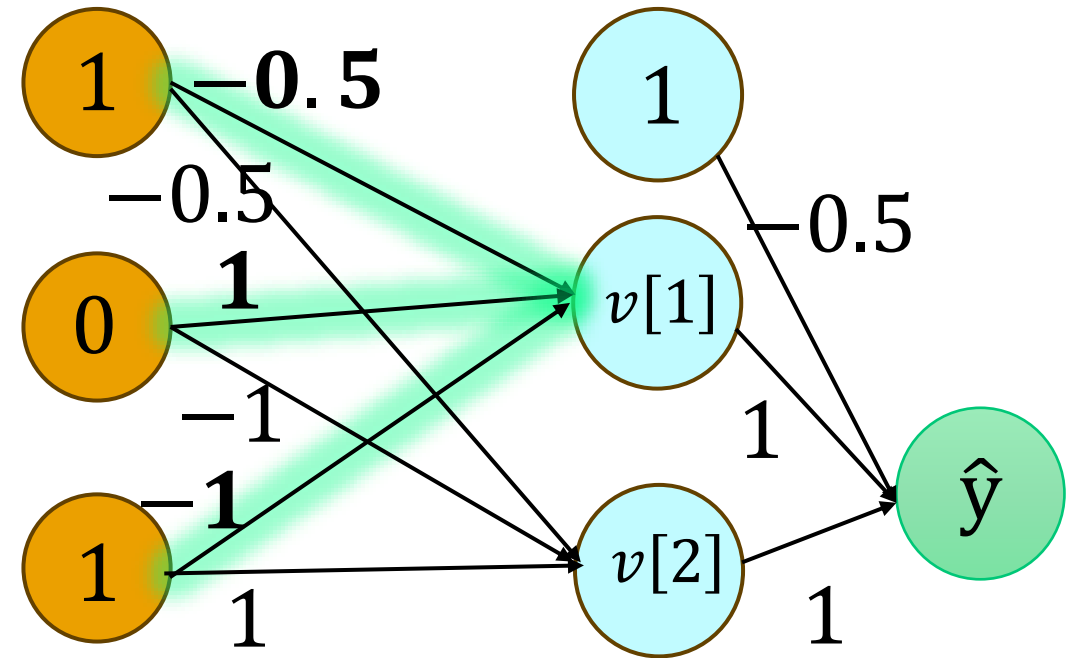
- Weights:

- $v[1] = g(-0.5 + x[1] - x[2])$
 - $[-0.5, 1, -1]$
- $v[2] = g(-0.5 - x[1] + x[2])$
 - $[-0.5, -1, 1]$
- $y = g(-0.5 + v[1] + v[2])$
 - $[-0.5, 1, 1]$



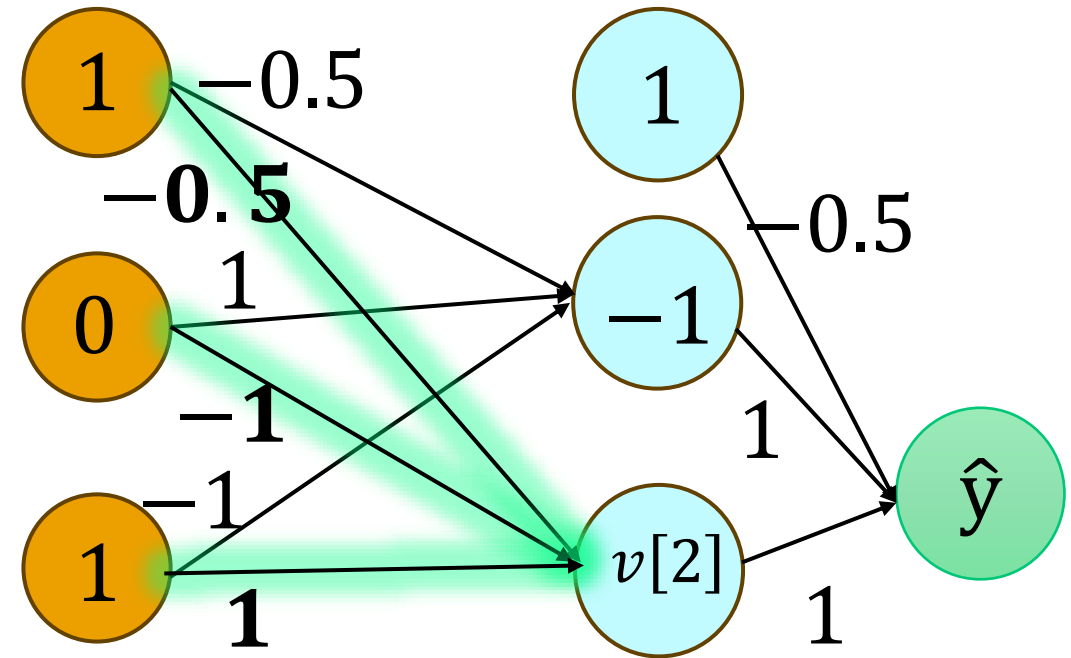
Back to XOR

- Example with $x[1] = 0, x[2] = 1$
- $v[1] = \text{sign}(-0.5 \cdot 1 + 1 \cdot 0 - 1 \cdot 1)$
 $= \text{sign}(-0.5 - 1) = -1$



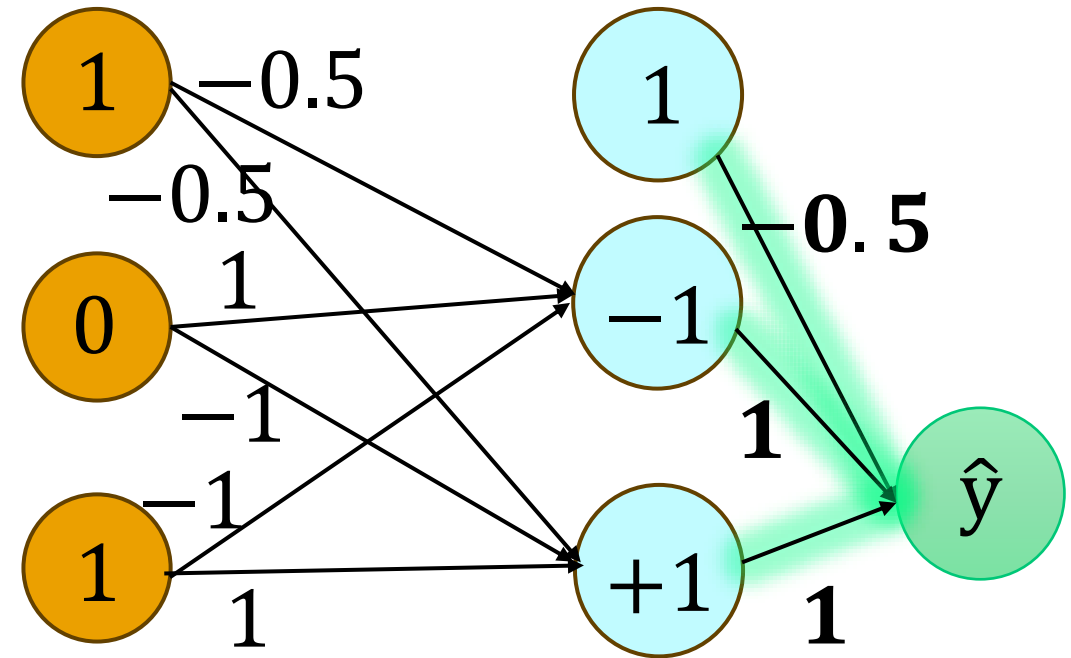
Back to XOR

- Example with $x[1] = 0, x[2] = 1$
- $v[1] = \text{sign}(-0.5 \cdot 1 + 1 \cdot 0 - 1 \cdot 1)$
 $= \text{sign}(-0.5 - 1) = -1$
- $v[2] = \text{sign}(-0.5 \cdot 1 - 1 \cdot 0 + 1 \cdot 1)$
 $= \text{sign}(-0.5 + 1) = +1$



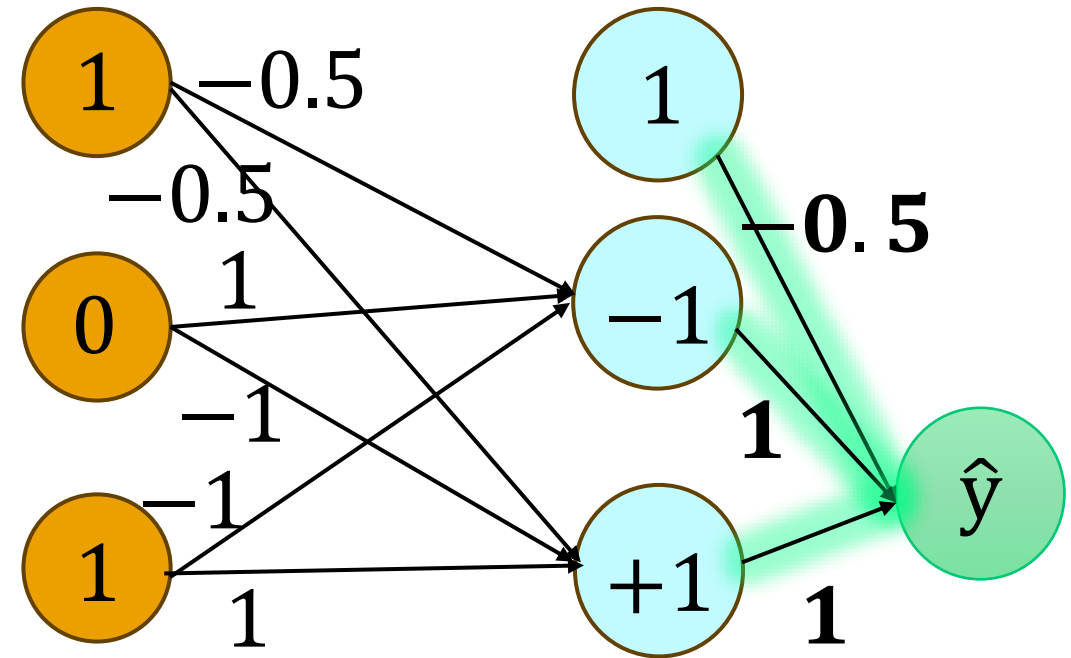
Back to XOR

- Example with $x[1] = 0, x[2] = 1$
- $v[1] = \text{sign}(-0.5 \cdot 1 + 1 \cdot 0 - 1 \cdot 1)$
 $= \text{sign}(-0.5 - 1) = -1$
- $v[2] = \text{sign}(-0.5 \cdot 1 - 1 \cdot 0 + 1 \cdot 1)$
 $= \text{sign}(-0.5 + 1) = +1$
- $y = \text{sign}(-0.5 \cdot 1 + 1 \cdot 0 + 1 \cdot 1)$
 $= \text{sign}(-0.5 - 0 + 1) = +1$
- Final output: +1



2 Layer NN

- With only 2 layers (input, hidden), you can represent any function!
 - As long as you have enough nodes in your hidden layer



Deep Learning

- Networks are generally considered “deep” with 2+ hidden layers
- However, deep networks often have many more than 2 hidden layers
 - GPT-3 has 96!
 - And 175B parameters (weights)!

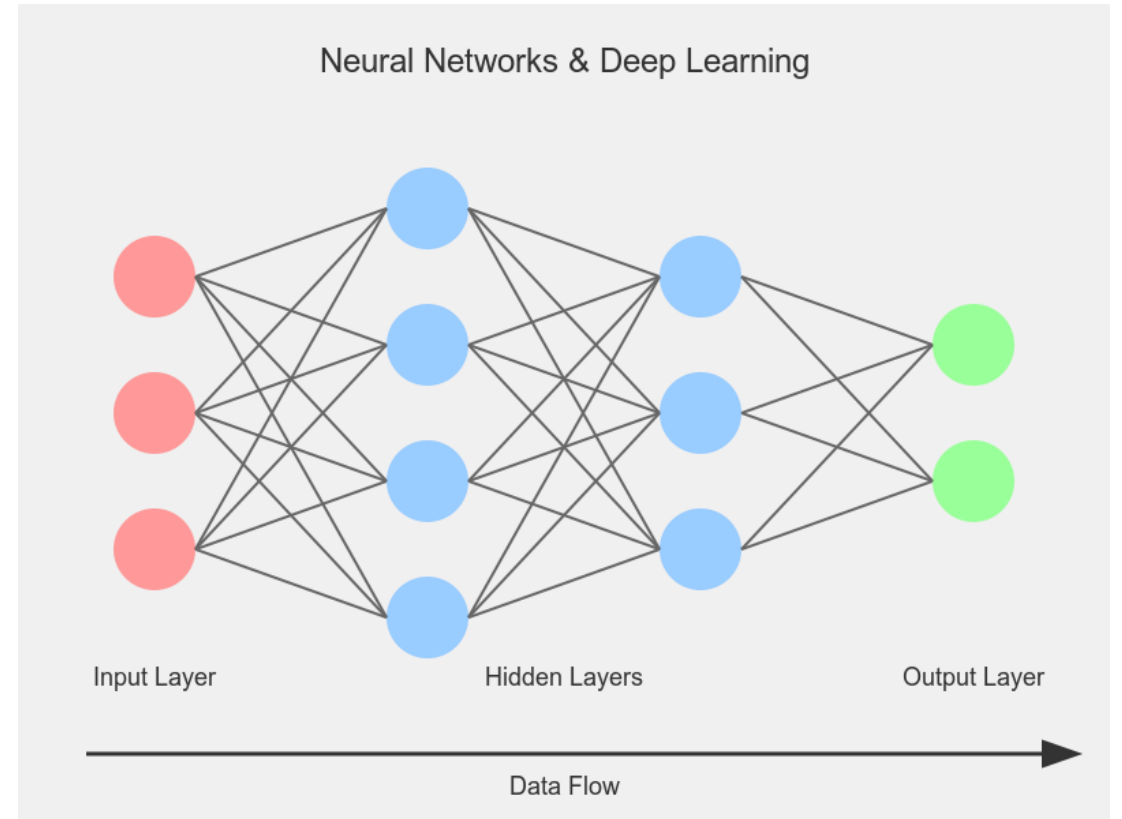


Image from <https://medium.com/@mustaphaliaichi/neural-networks-and-deep-learning-a-comprehensive-introduction-092449336c1f>

Break

Get some water, stretch your legs, chat, be back in 10

Activation Functions

- We have only shown the $\text{sign}(\)$ activation function so far
- This is simple, but rarely used in practice as the sign function
 - Is not differentiable
 - Has no notion of **confidence** (probability correct)
- Question: what is one function we've used before to replace the sign function?
 - Hint: when transforming our linear regression model into something we can use for classification

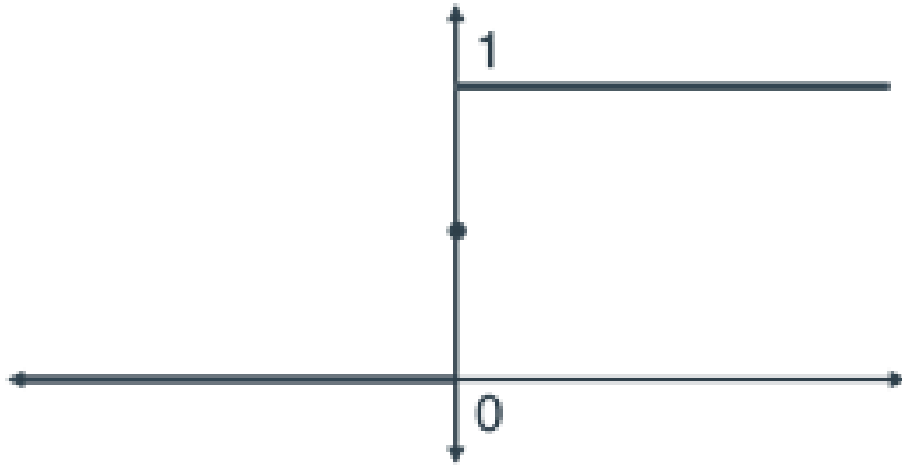
Activation Functions

- We have only shown the $\text{sign}(\quad)$ activation function so far
- This is simple, but rarely used in practice as the sign function
 - Is not differentiable
 - Has no notion of **confidence** (probability correct)
- Question: what is one function we've used before to replace the sign function?
 - Hint: when transforming our linear regression model into something we can use for classification
- Answer: logistic/sigmoid functions (like for logistic regression)!

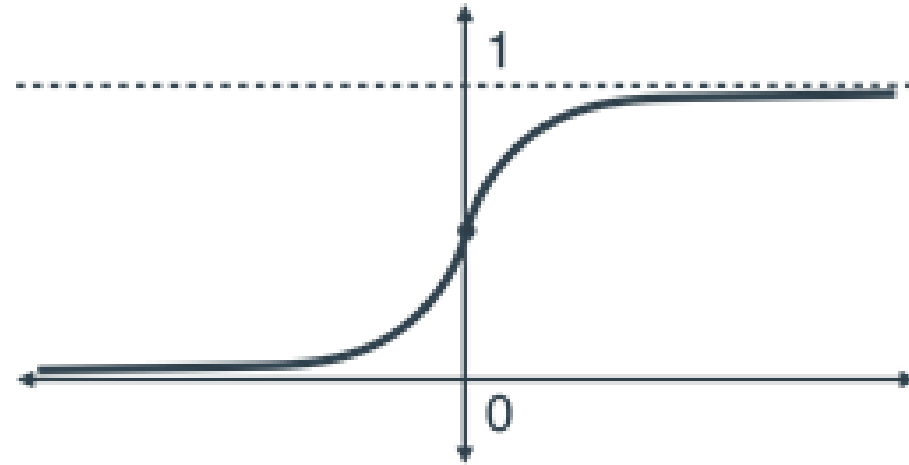
Activation Functions

- Sign vs Sigmoid function reminder:

Step function
(discrete)

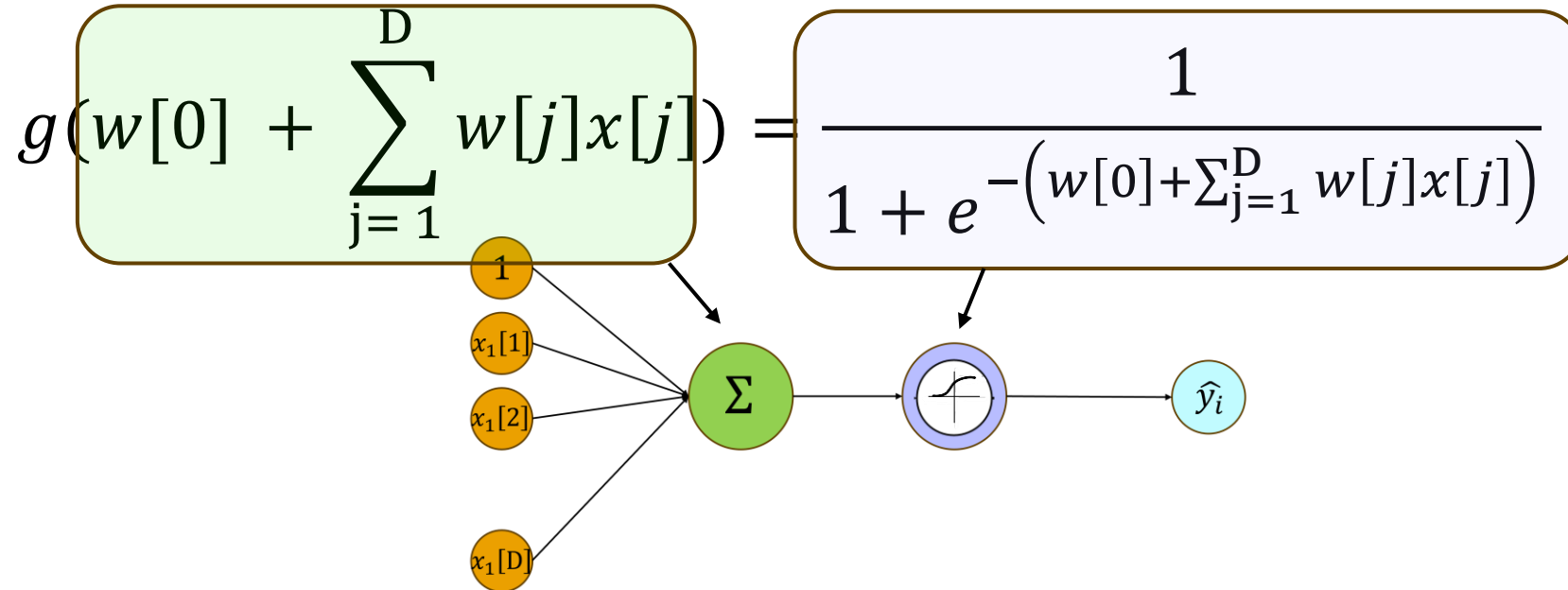


Sigmoid function
(continuous)



Activation Functions

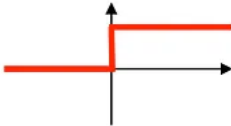
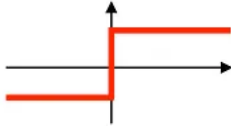
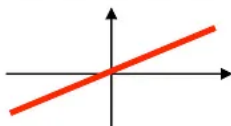
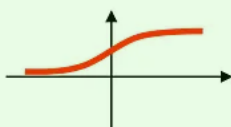
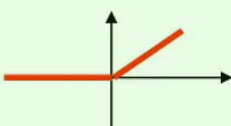
- Sigmoid activation function:



- For simplicity, we can call $w[0] + \sum_{j=1}^D w[j]x[j]$ the **score** of a node
 - The **activation function** takes the **score** as input
 - So $g = \frac{1}{1 + e^{-(\text{Score}(\mathbf{x}))}}$

Common Activation Functions

- Sigmoid
 - $\frac{1}{1+e^{-(\text{Score}(x))}}$
- Hyperbolic tangent (tanh)
 - $\tanh(\text{Score}(x))$
- Rectified linear unit (ReLU)
 - $\max(0, \text{Score}(x))$
- Leaky ReLU
 - $\begin{cases} 0.01(\text{Score}) & \text{for } \text{Score} < 0 \\ \text{Score} & \text{for } \text{Score} \geq 0 \end{cases}$
- Softplus
 - $\log(1 + e^{\text{Score}})$

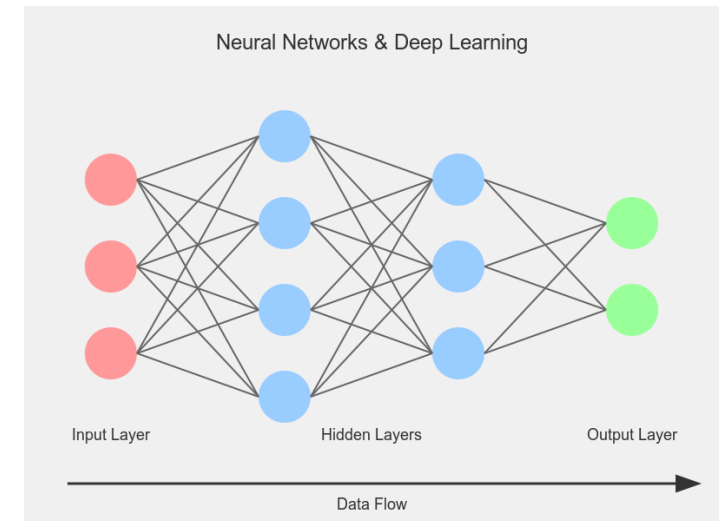
Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

Image from
<https://medium.com/@sreeku.ralla/activation-functions-relu-vs-leaky-relu-b8272dc0b1be>

Neural networks for Classification/Regression

- Classification
 - Output layer has **one node per class**
 - Can use the **softmax logistic function** (like multinomial logistic regression) to turn scores into probabilities
 - Output the node with the highest score as prediction
- Regression
 - Output is generally one node that outputs a single number
 - No activation function on last layer
 - Keeps output as a continuous unbounded number



Learning Parameters for NNs

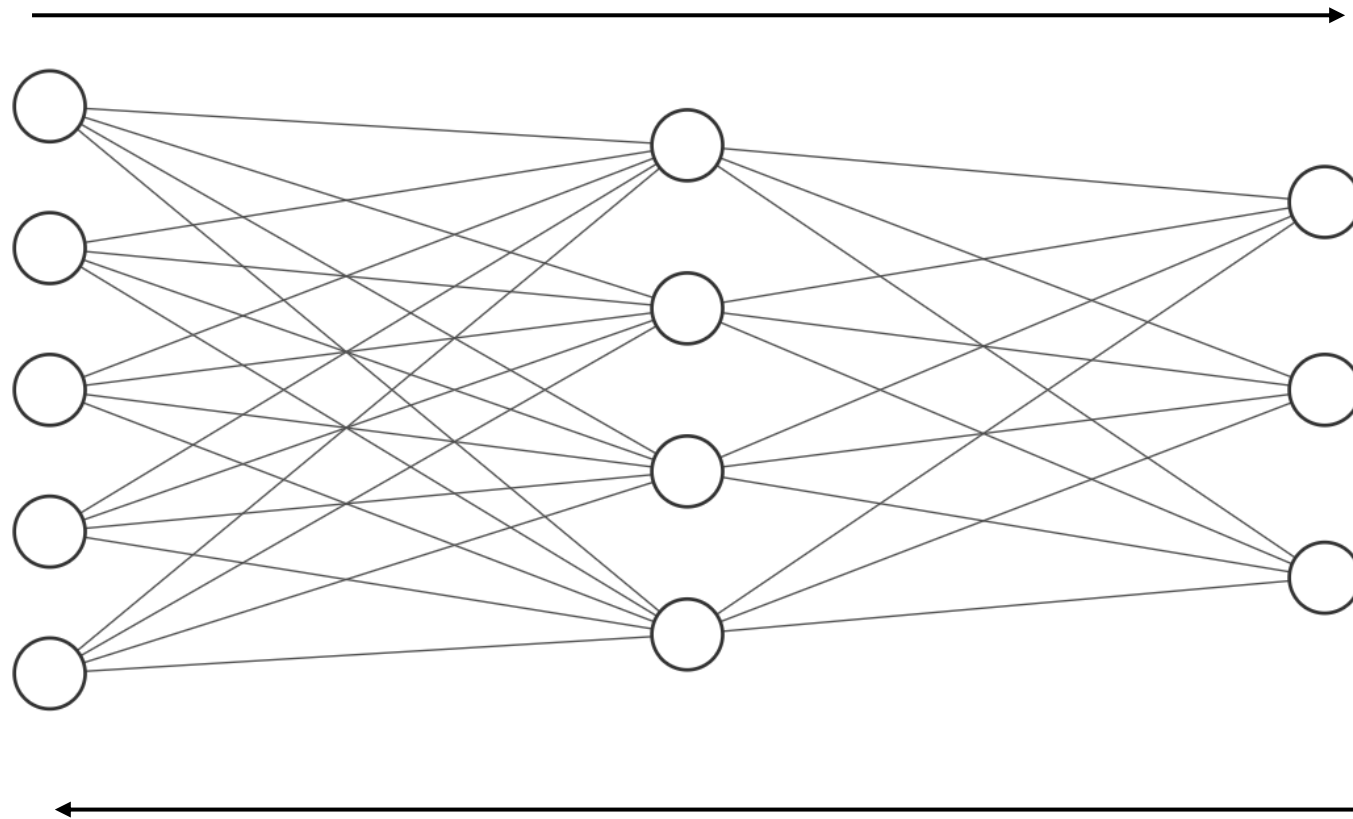
- How do we actually learn the weights for a neural network?
- First we need some measure of **loss**
- Question: what might we use for:
 - Regression?
 - Classification?

Learning Parameters for NNs

- How do we actually learn the weights for a neural network?
- First we need some measure of **loss**
- Question: what might we use for:
 - Regression?
 - MSE, RMSE
 - Classification?
 - Cross Entropy Loss
- And we'll use everyone's favorite solving algorithm, gradient descent!

Learning Parameters for NNs

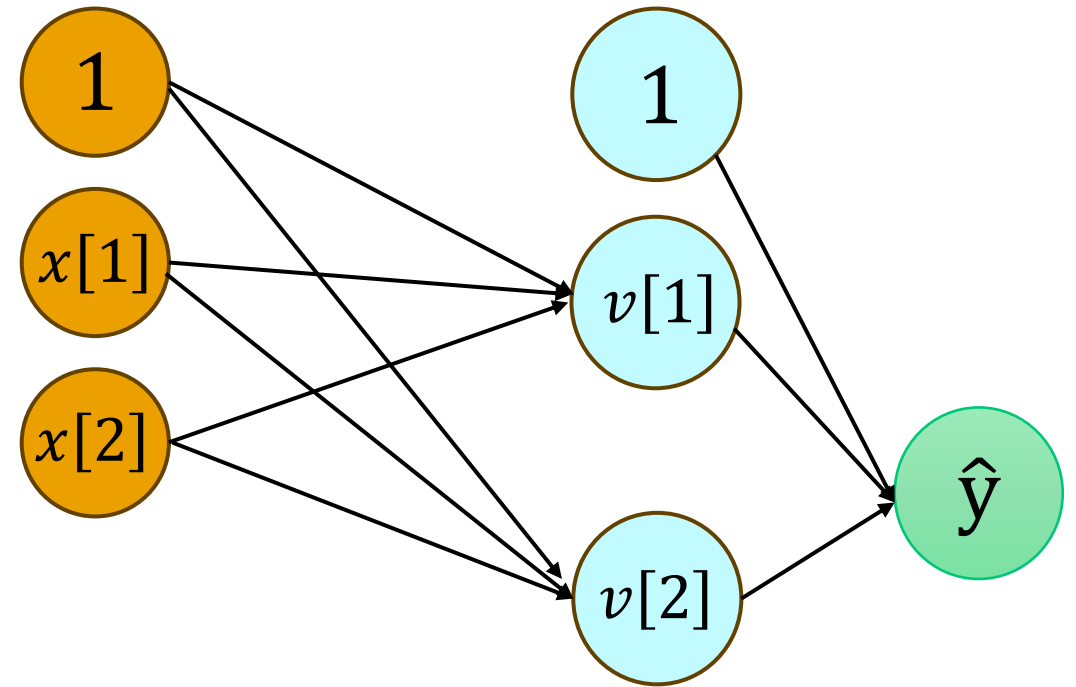
Forward Pass (calculating output from input)



Backpropagation (updating all weights using gradient descent)

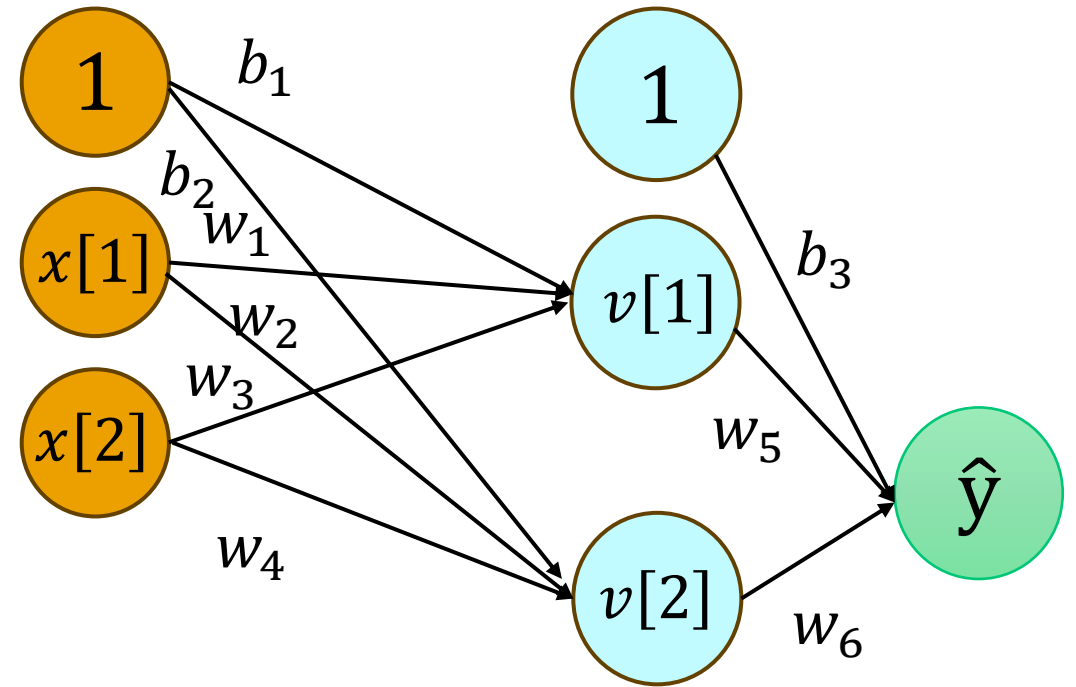
Example: Learning Parameters for NNs

- Question: how many weights to learn?



Example: Learning Parameters for NNs

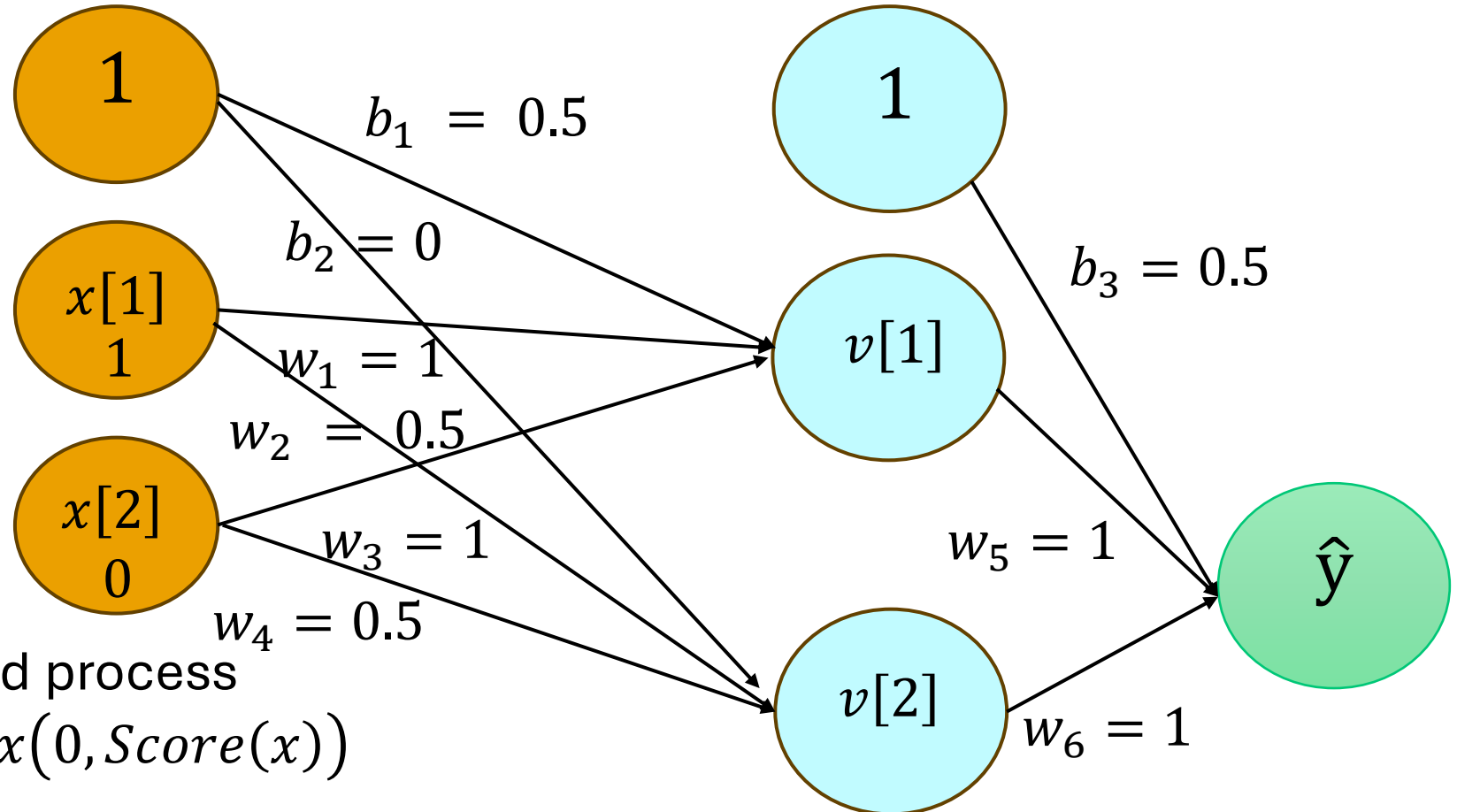
- Question: how many weights to learn?
- Each arrow has a weight
 - 9 weights to learn
 - (6 weights, 3 bias terms)
- This is a regression network
 - We'll use ReLU on hidden layer
 - No activation on output



Example: Learning Parameters for NNs

- First training example: $x = [1,0], y = 2$
- Initial weights:

- $b_1 = 0.5$
- $b_2 = 0$
- $w_1 = 1$
- $w_2 = 0.5$
- $w_3 = 1$
- $w_4 = 0.5$
- $b_3 = 0.5$
- $w_5 = 1$
- $w_6 = 1$

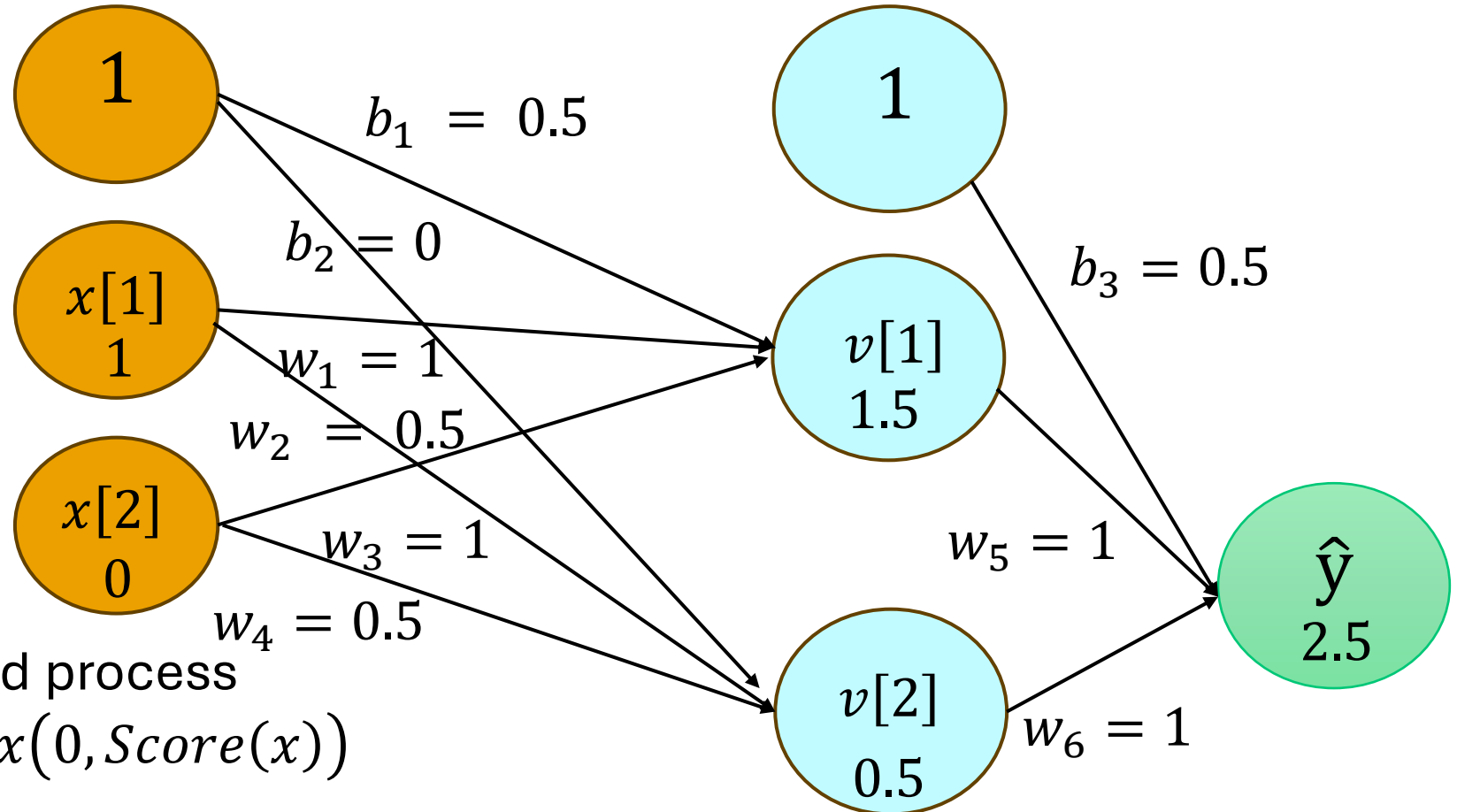


- First, do the feed forward process
- Reminder: $\text{ReLU} = \max(0, \text{Score}(x))$

Example: Learning Parameters for NNs

- First training example: $x = [1,0], y = 2$
- Initial weights:

- $b_1 = 0.5$
- $b_2 = 0$
- $w_1 = 1$
- $w_2 = 0.5$
- $w_3 = 1$
- $w_4 = 0.5$
- $b_3 = 0.5$
- $w_5 = 1$
- $w_6 = 1$



- First, do the feed forward process
- Reminder: $\text{ReLU} = \max(0, \text{Score}(x))$
- Output: 2

Example: Learning Parameters for NNs

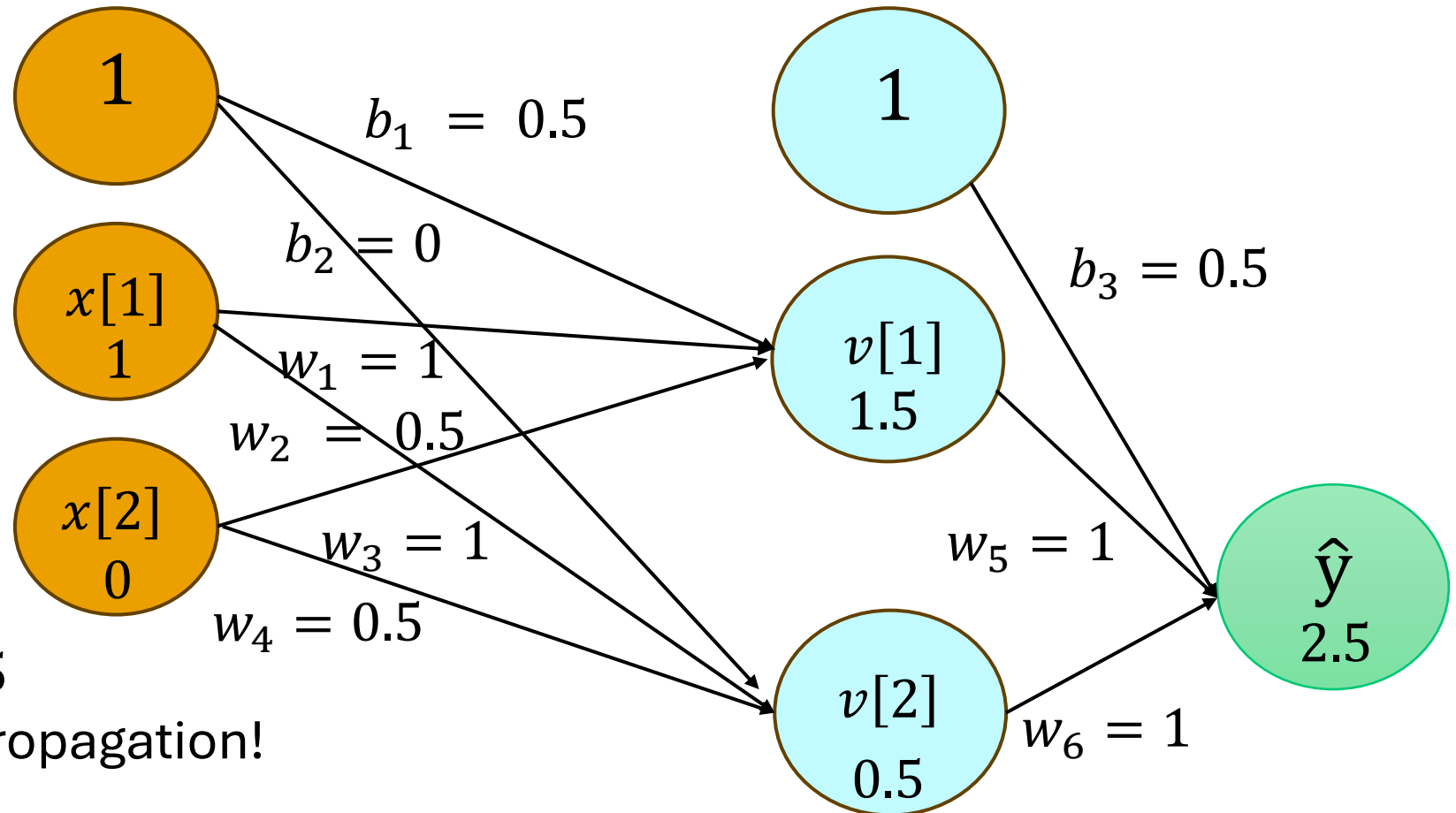
- First training example: $x = [1, 0]$, $y = 2$

- Initial weights:

- $b_1 = 0.5$
- $b_2 = 0$
- $w_1 = 1$
- $w_2 = 0.5$
- $w_3 = 1$
- $w_4 = 0.5$
- $b_3 = 0.5$
- $w_5 = 1$
- $w_6 = 1$

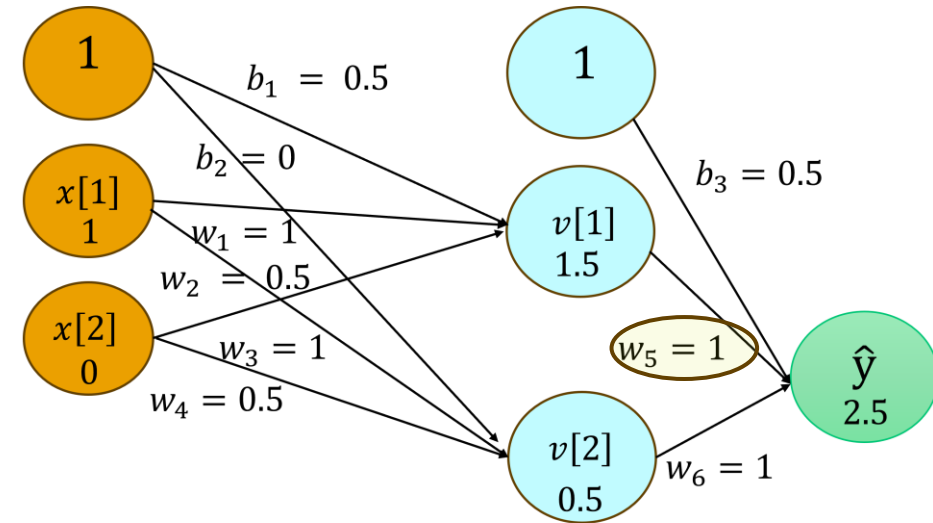
- MSE: $(2.5 - 2)^2 = 0.25$

- Now we perform backpropagation!



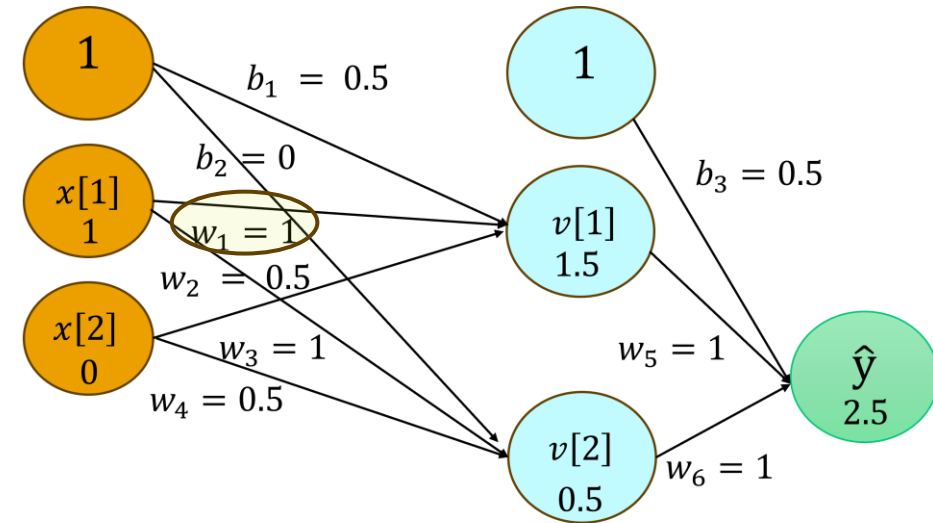
Example: Learning Parameters for NNs

- MSE: $(2.5 - 2)^2 = 0.25$
- $L = (b_3 + w_5 v[1] + w_6 v[2] - y)^2$
- Let's start with updating w_5
- Just like regular gradient descent
- Take the gradient of the loss function
- $\frac{\partial L}{\partial w_5} = 2v[1](b_3 + w_5 v[1] + w_6 v[2] - y)$
 - Chain rule
- $v[1] = 1.5, (b_3 + w_5 v[1] + w_6 v[2] - y) = 0.5$
- $\frac{\partial L}{\partial w_5} = 1.5$
- $w_5 = w_5 - \eta \frac{\partial L}{\partial w_5}$
- Let's say $\eta = 0.01$ (learning rate)
- $w_5 = w_5 - \eta \frac{\partial L}{\partial w_5} = 1 - 0.01(1.5) = 0.985$



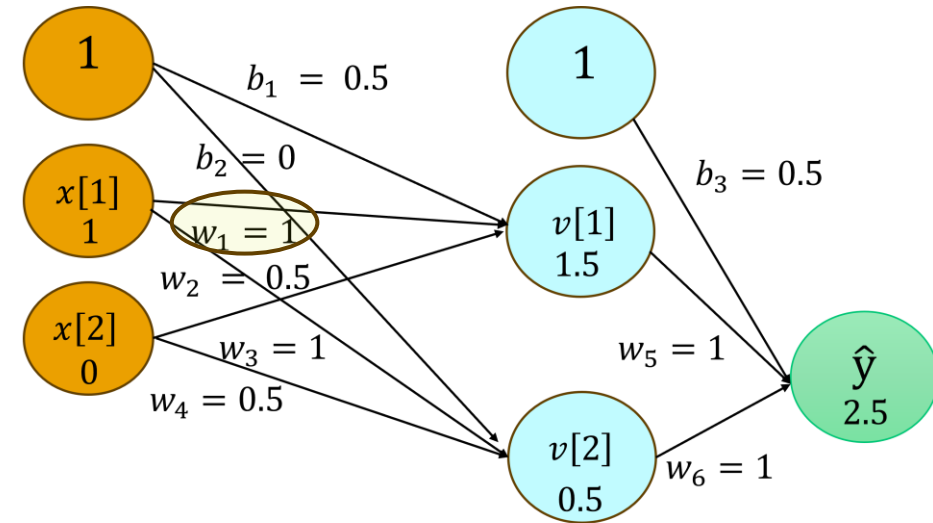
Example: Learning Parameters for NNs

- MSE: $(2.5 - 2)^2 = 0.25$
- Calculating the outer weights is just like regular gradient descent or perceptron updates
- What about a weight farther back?
- Now that we've updated w_5 , let's try updating w_1



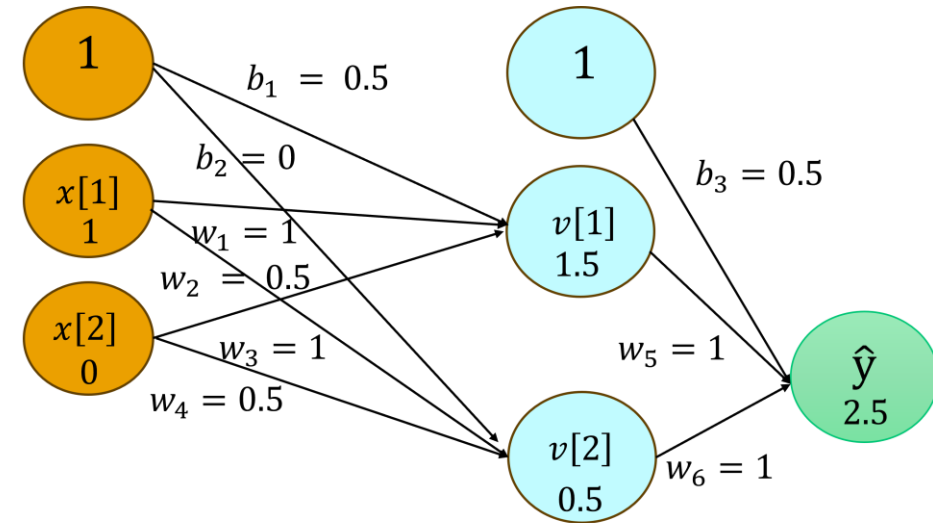
Example: Learning Parameters for NNs

- MSE: $(2.5 - 2)^2 = 0.25$
- $L = (b_3 + w_5 v[1] + w_6 v[2] - y)^2$
- Updating w_1
- $\frac{\partial L}{\partial w_1} = ?$
- No w_1 in L !
- We'll express $v[1]$ as a function of w_1 and use the chain rule to take the derivative
- $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial v[1]} \frac{\partial v[1]}{\partial w_1}$
- $\frac{\partial L}{\partial v[1]} = 2w_5(b_3 + w_5 v[1] + w_6 v[2] - y) = 1$
- $v[1] = \begin{cases} 0 & \text{if Score} < 0 \\ \text{Score} & \text{if Score} \geq 0 \end{cases}$



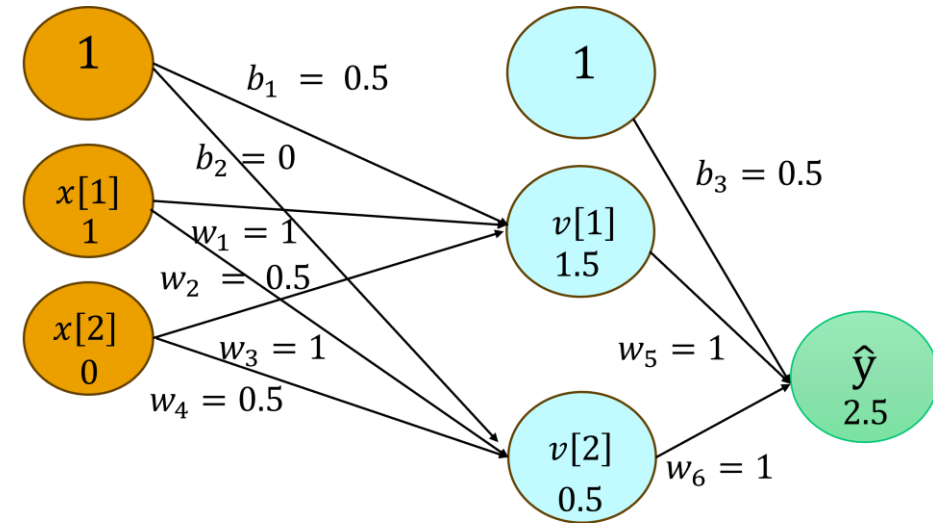
Example: Learning Parameters for NNs

- $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial v[1]} \frac{\partial v[1]}{\partial w_1}$
- $\frac{\partial L}{\partial v[1]} = 2w_5(b_3 + w_5v[1] + w_6v[2] - y) = 1$
- $v[1] = \begin{cases} 0 & \text{if Score} < 0 \\ \text{Score} & \text{if Score} \geq 0 \end{cases}$
- We know $\text{Score} = w_1x[1] + w_2x[2] + b_1 \geq 0$ (1.5) from calculation of $v[1]$
- So piecewise,
 - $\frac{\partial v[1]}{\partial w_1} = x[1] = 1$
- $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial v[1]} \frac{\partial v[1]}{\partial w_1} = 1 \cdot 1 = 1$
- $w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} = 1 - 0.01(1) = 0.99$



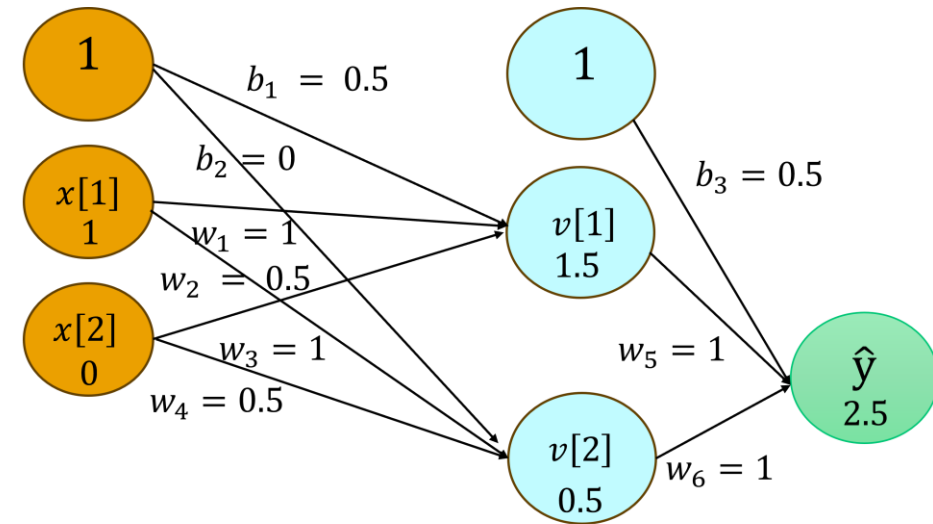
Example: Learning Parameters for NNs

- You can update the other weights in the same way!
- In future courses, you'll learn how to use tools like PyTorch to calculate these gradients for you and handle large amounts of data



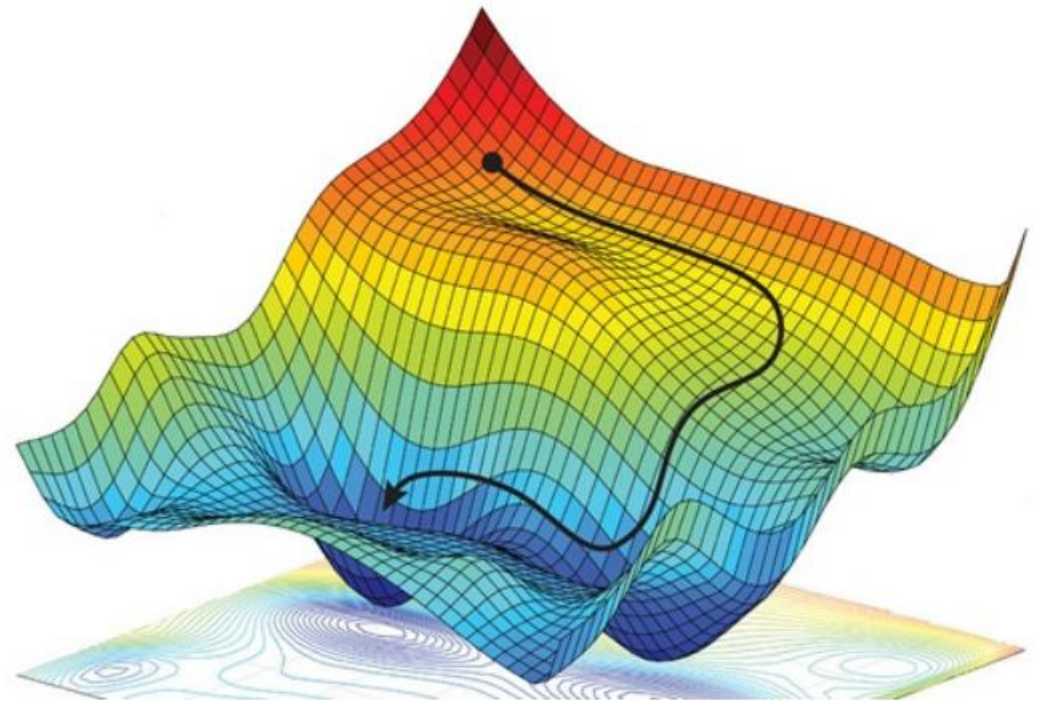
Batches

- Speaking of large amounts of data, it would be very computationally expensive to update all weights for every single datapoint
- Instead, you often break up your dataset in **batches**, train on everything in one batch, then update weights based on the overall loss
- Processing each batch once often isn't enough
 - An iteration that goes over every batch once is called an **epoch**
 - **Epoch**: one step of training NN on the entire training set



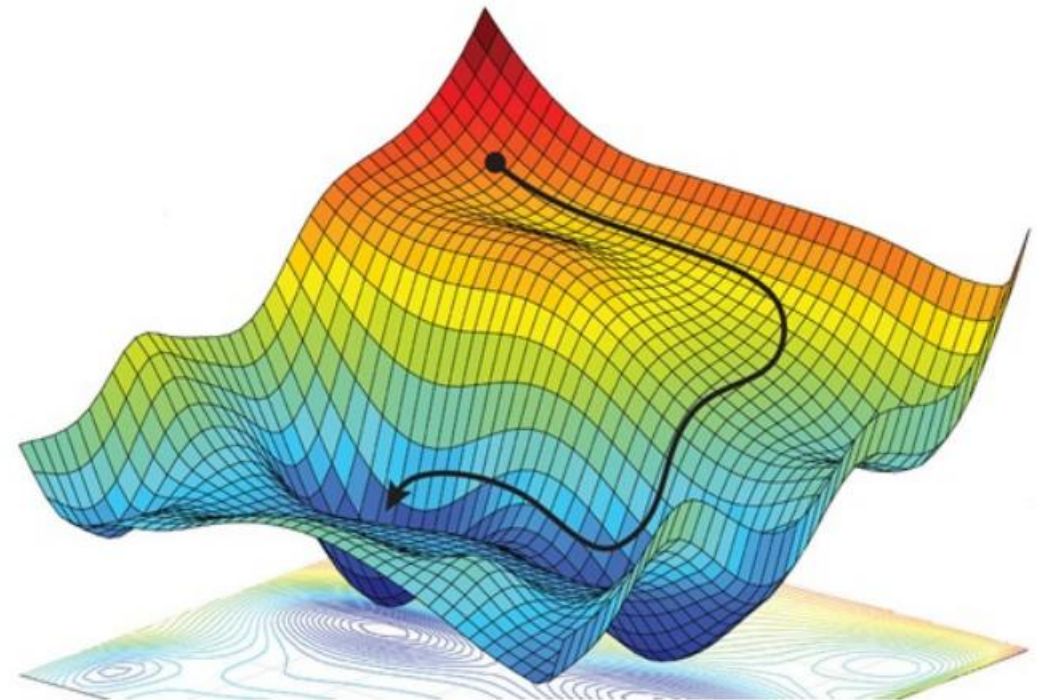
NN Training Issues

- In general, NNs do not create convex loss functions
- Question: what does this mean for backpropagation?



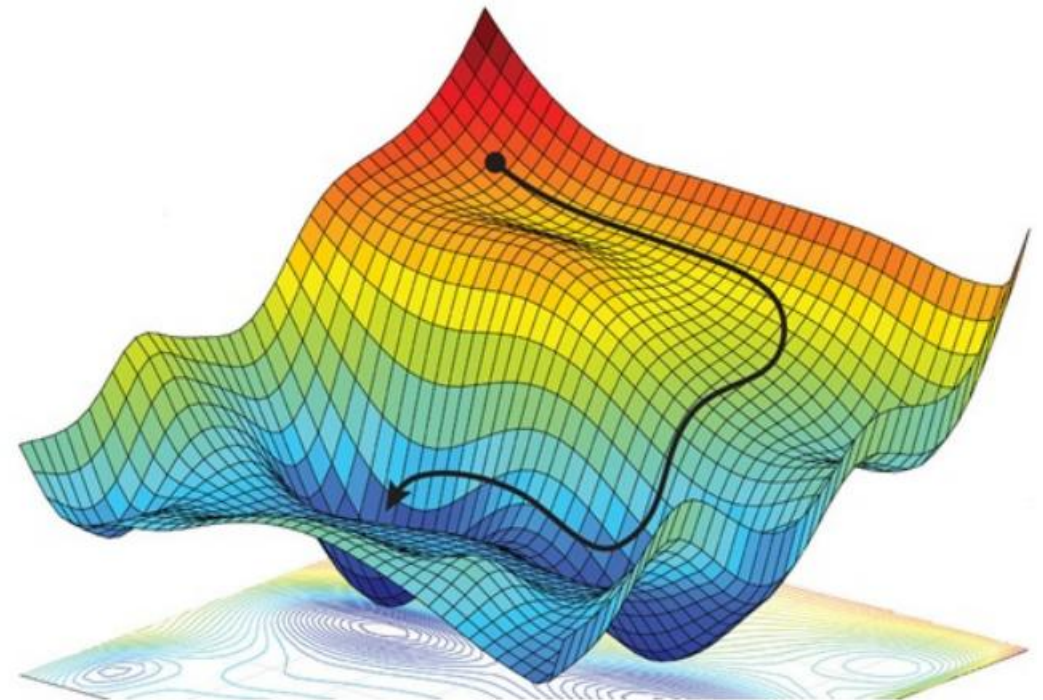
NN Training Issues

- In general, NNs do not create convex loss functions
- Question: what does this mean for backpropagation?
- It will converge to **local optima**, not **global**
- Thus, weight initialization can heavily impact the final result
- People often do random initialization
- You can also change the learning rate to reduce the likelihood of getting stuck in local minima
 - Decreasing learning rate over time
 - Oscillating between a min and max value to “jumpstart” out of local minima
 - And others!



NN Training Issues

- Hyperparameters for NNs
 - # of hidden layers
 - # of hidden neurons in **each layer**
 - Each layer can have a different # of neurons
 - Activation function
 - Each layer can have a different activation function
 - Gradient Descent learning rate
 - Batch size
 - # of epochs to train
 - ...and more
- NNs are unfortunately not very interpretable, and we're still learning how these hyperparameters impact performance

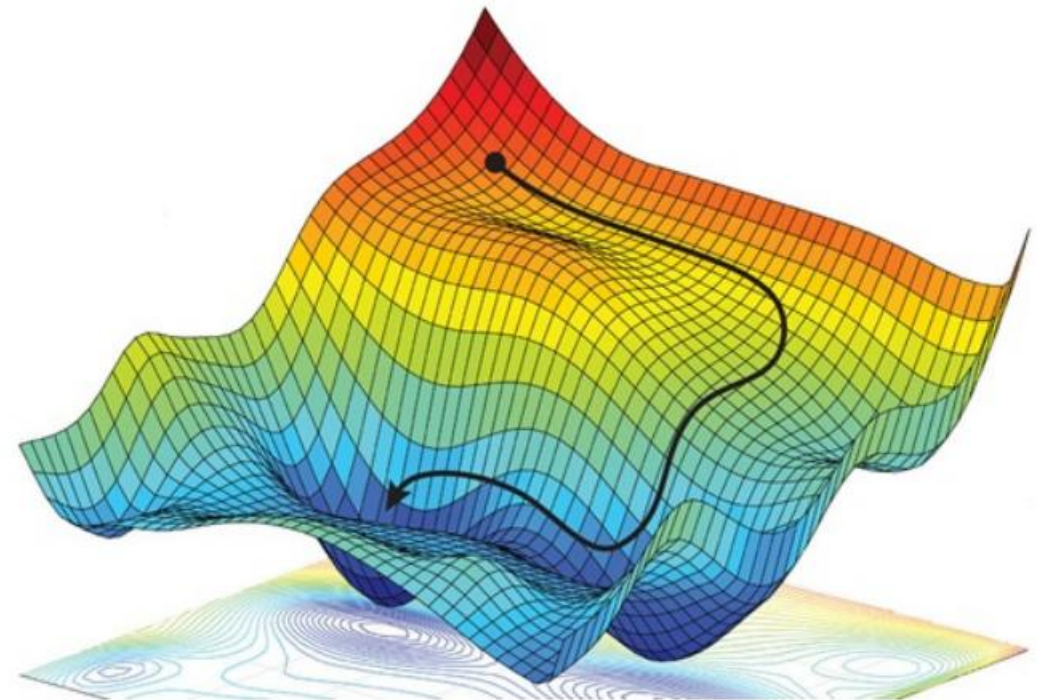


NN Training Issues

- Choosing activation functions
 - Sigmoid
 - Was popular for quite a while, but has fallen out of favor
 - **Vanishing gradients:** when weights get large, the gradients can get tiny, and thus updates become tiny
 - Not zero-centered (all updates can become positive, making convergence slower)
 - Still used often on output layer (“softmax” for class probability)
 - Tanh
 - Has same vanishing gradient issue as Sigmoid
 - Is zero-centered
 - ReLU
 - Very popular choice
 - Careful with learning rates as neurons can “die off”
 - If input to ReLU is always negative, output will always be 0 and contribute nothing

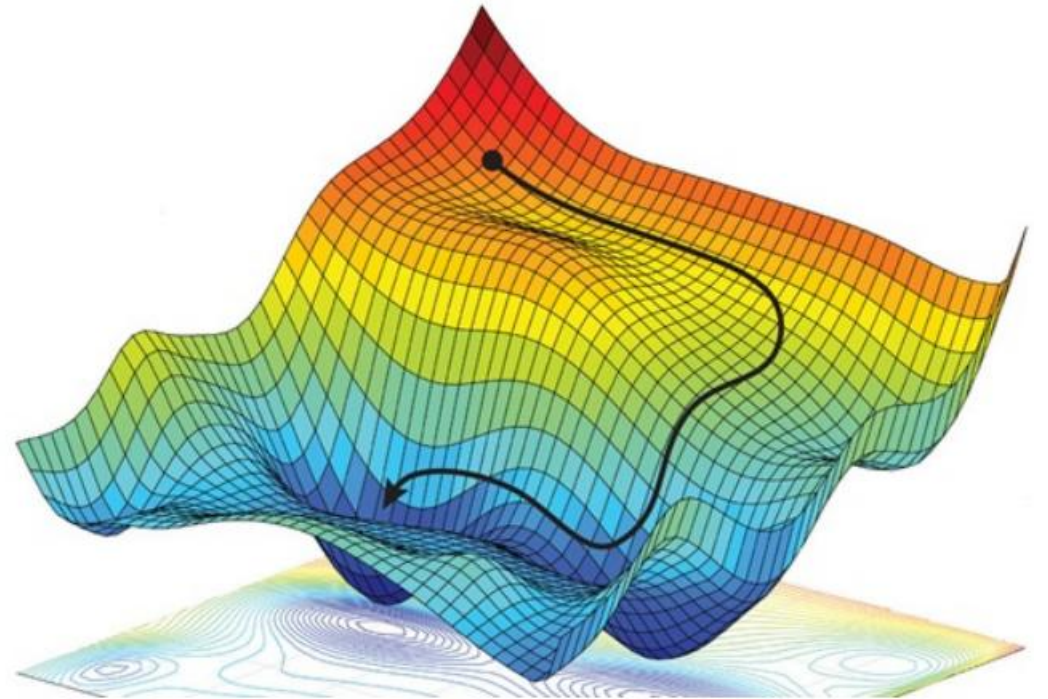
NN Training Issues

- Neural networks (especially deep learning) often overfit
- Question: how is this connected to why deep learning needs so much data to learn?



NN Training Issues

- Neural networks (especially deep learning) often overfit
- Question: how is this connected to why deep learning needs so much data to learn?
- Increasing the amount of data can increase bias (reduce overfitting)!



Break

Get some water, stretch your legs, chat, be back in 10



Demo

Discussion

People sometimes suggest that, since deep learning is modeled on the human brain, it is the best way to achieve human-level intelligence in AI. Do you agree or disagree? Why? Do you think human-level intelligence from machines is possible?

Discussion

Foundational models are large-scale AI models (like deep networks) trained on vast amounts of data and applied to a wide range of tasks. What might be the pros and cons of using foundational models vs specialized application models?